



年中复盘会（2/2）—— 25年上半年中大厂高频面试总结

1. 往期部分专题

- [📖 以终为始，重回前端 -- Web Development Trend](#)
- [📖 以终为始，重回前端 --Typescript： JavaScript with syntax for types](#)
- [📖 以终为始，重回前端 -- Awesome JavaScript & Design Patterns](#)
- [📖 以终为始，重回前端（特别篇） -- New React 19 Features You Should Know & How to Upgrade](#)
- [📖 以终为始，重回前端 -- 构建高性能 Vue应用：揭秘 Module Federation 微前端架构的核心技术](#)
- [📖 以终为始，重回前端 -- 揭秘大厂工程化内核，从软件工程视角剖析编译时、运行时及打包构建三板斧](#)
- [📖 以终为始，重回前端 -- 字节前端专家揭秘大厂React最佳实践，手把手从0打造一个开源社区前沿的组件库](#)
- [📖 2025 金三银四面试突击早鸟课 -- 一线25K 前端P6岗的候选人能力要求 & 画像解析](#)
- [📖 2025 金三银四面试突击早鸟课第二弹——字节T2-1岗位面试真题解析](#)
- [📖 2025 金三银四面试突击早鸟课第三弹——手写对标一线城市25K前端P6岗的满分简历](#)
- [📖 金三银四面试突击ALL IN ONE 第三篇 —— 大厂P6级模拟面试来啦](#)
- [📖 金三银四面试突击ALL IN ONE 第四篇 —— 25年金三银四面试高频踩坑解析](#)
- [📖 金三银四面试突击ALL IN ONE 第五篇 —— 金三银四面试利器之前端技术架构设计实操](#)
- [📖 金三银四面试突击ALL IN ONE 第六篇 —— 金三银四大厂真实面试原题解析](#)
- [📖 618专题--大促搭投实践&高可用性保障](#)
- [📖 618年中复盘会（1/2）—— 25年上半年中大厂高频面试总结](#)

2. 基础真题解析

2.1 Call apply bind new

Call

Code block

```
1  Function.prototype.call2 = function(context,
```

apply

Code block

```
1  Function.prototype.apply2 = function(context,
```

```

    ...args) {
2      // 判断是否是undefined和null
3      if (typeof context === 'undefined' ||
        context === null) {
4          context = window
5      }
6      let fnSymbol = Symbol()
7      context[fnSymbol] = this
8      let fn = context[fnSymbol](...args)
9      delete context[fnSymbol]
10     return fn
11 }

```

```

    args) {
2      // 判断是否是undefined和null
3      if (typeof context === 'undefined' ||
        context === null) {
4          context = window
5      }
6      let fnSymbol = Symbol()
7      context[fnSymbol] = this
8      let fn = context[fnSymbol](...args)
9      delete context[fnSymbol]
10     return fn
11 }

```

Bind

Code block

```

1  Function.prototype.bind2 = function (context)
   {
2
3      if (typeof this !== "function") {
4          throw new
Error("Function.prototype.bind - what is
trying to be bound is not callable");
5      }
6
7      var self = this;
8      var args =
Array.prototype.slice.call(arguments, 1);
9
10     var fNOP = function () {};
11
12     var fBound = function () {
13         var bindArgs =
Array.prototype.slice.call(arguments);
14         return self.apply(this instanceof
fNOP ? this : context, args.concat(bindArgs));
15     }
16
17     fNOP.prototype = this.prototype;
18     fBound.prototype = new fNOP();
19     return fBound;
20 }

```

Code block

```

1  function objectFactory() {
2      var obj = new Object(),
3      Constructor = [].shift.call(arguments);
4      obj.__proto__ = Constructor.prototype;
5      var ret = Constructor.apply(obj,
arguments);
6      return typeof ret === 'object' ? ret :
obj;
7
8  };

```

New

2.2 行内元素 块元素

块级元素

- 特性: 块级元素总是从新的一行开始, 并且其宽度默认情况下会尽可能地延伸到其父容器的宽度, 即“独占一行”
- `<div>`, `<h1>` 到 `<h6>`, `<p>`, `<ul>`, `<ol>`, `<li>`, `<form>`, `<table>` 等都是块级元素。
- 布局行为: 块级元素可以设置宽度、高度、内外边距等属性, 并且可以包含其他块级元素或行内元素。

行内元素

行内元素 块元素 继承属性

- 文本相关的属性:
 - `font` (字体): 包括字体族、大小、样式、粗细等。
 - `color` (文本颜色): 文本的颜色。
 - `letter-spacing` (字符间距): 字符之间的间隔。
 - `word-spacing` (字词间距): 字词之间的间隔。
 - `line-height` (行高): 行与行之间的距离。
 - `text-align` (文本对齐): 文本在容器内的对齐方式。
 - `text-indent` (首行缩进): 段落首行文本的缩进。
 - `text-transform` (文本转换): 文本大小写转换。

- 特性: 行内元素不独占一行，它们会在同一行内显示，根据内容自动调整宽度和高度。
- `<span>`，`<a>`，`<img>`，`<strong>`，`<em>`，`<b>`，`<i>`，`<input>`，`<button>` 等都是行内元素。
- 布局行为: 行内元素不能设置宽度和高度，只能设置内边距（padding），外边距（margin）的上下方向无效，左右方向有效。行内元素内部只能包含行内元素，不能包含块级元素。

- `white-space` (空白处理): 控制空白字符的处理方式。
- `unicode-bidi` (文本方向): 控制文本的字符方向。
- `direction` (文本方向): 控制文本的书写方向。
- 可见性:
 - `visibility`: 控制元素是否可见。
 - `cursor`: 定义鼠标悬停在元素上时的光标形状。
- 列表相关的属性:
 - `list-style`: 列表项的样式，包括类型、图片和位置。
 - `list-style-type`: 列表项标记的类型。
 - `list-style-position`: 列表项标记的位置。
 - `list-style-image`: 列表项标记的图片。
- 表格相关的属性:
 - `border-collapse`: 控制表格边框的合并方式。
 - `border-spacing`: 控制表格单元格之间的间距。
 - `caption-side`: 控制表格标题的位置。
 - `empty-cells`: 控制是否显示空单元格的边框。
 - `table-layout`: 控制表格布局算法。

2.3 CSS样式优先级

内联样式> ID选择器> 类选择器、属性选择器、伪类选择器> 标签选择器、伪元素选择器。通用选择器、子选择器、相邻同胞选择器优先级最低，都为0

1. 内联样式(Inline styles): 直接写在HTML元素上的 `style` 属性，例如 `<div style="color: red;">`，优先级最高，权重值为1000。
2. ID 选择器: 使用 `#` 符号定义的，例如 `#content`，优先级第二，权重值为100。
3. 类选择器(Class selectors), 属性选择器(Attribute selectors), 伪类选择器(Pseudo-class selectors): 使用 `.` 符号定义的，例如 `.container`，或者使用 `[]` 定义的属性选择器，例如 `[type="text"]`，以及冒号开头的伪类选择器，例如 `:hover`，优先级第三，权重值都为10。
4. 标签选择器(Type selectors), 伪元素选择器(Pseudo-element selectors): 使用HTML标签名称定义的，例如 `div`，`p`，或者 `::before` 这样的伪元素选择器，优先级最低，权重值都为1。
5. 通用选择器 (`*`), 子选择器 (`>`), 相邻同胞选择器 (`+`): 这些选择器优先级最低，都为0。

2.5 BFC

Block formatting context 块级格式化上下文

指的是一个独立的区域，在这个区域内，块级盒子(block-level box) 按照一定的规则进行布局，并且这个区域的布局不受外部的影响。简单来说，BFC就像一个独立的容器，里面的元素布局

2.4 原型链

原型链（prototype chain）是实现对象继承的核心机制。

每个对象都有一个内部属性 `[[Prototype]]`（通常通过 `proto` 访问），指向它的原型对象。当访问一个对象的属性时，如果该对象自身不存在这个属性，JavaScript引擎就会沿着 `[[Prototype]]` 链向上查找，直到找到该属性或者到达链的末端（`null`）。这个由对象之间的 `[[Prototype]]` 关系构成的链条就叫做原型链。

2.6 Margin 重叠

1. 两个正数的外边距取最大的边距作为外边距。
2. 如果一个为正数，一个为负数，最终的外边距为两者之和。
3. 如果两个值都是负数的话，最终的外边距取绝对值最大的值。

规则与其他区域无关，而这个容器的高度计算也会将浮动元素考虑在内

BFC 的主要作用和特点：

- **隔离布局:**BFC 内部的块级元素布局不受外部的影响，反之亦然。
 - **清除浮动:**可以用来解决浮动元素导致的父元素高度塌陷问题。
 - **避免外边距折叠:**属于同一个BFC 的相邻块级元素的垂直方向外边距会发生重叠，而不同BFC 之间的元素则不会。
 - **计算高度:**BFC 的高度计算会将浮动元素也包含在内，避免了浮动元素撑开父元素高度的问题。
 - **独立渲染区域:**BFC 是一个独立的渲染区域，它内部的元素布局规则与其他区域无关，这使得我们可以更好地控制页面的布局。
- `float` 除了 `none` 以外的值
 - `overflow` 除了 `visible` 以外的值
 - `display: inline-block`
 - `display: table-cell`
 - `display: flex` / `display: inline-flex`
 - `display: grid` / `display: inline-grid`
 - `position: absolute` / `position: fixed`
 - `contain: layout`

2.7 Let const var

1. `var` : 函数作用域 (function-scoped) `let` / `const` : 块级作用域 (block-scoped)
2. `var` : 变量会被提升到函数或全局作用域的顶部，初始值为 `undefined` ； `let` / `const` : 也会提升，但不会被初始化（暂时性死区 TDZ； *ReferenceError*）
3. `var` : 在全局作用域声明时，会成为全局对象(`window`)的属性； `let` / `const` : 不会成为全局对象的属性

Code block

```
1 let x = 1;
2 console.log(window.x); // undefined
```

2.8 Symbol.Iterator

`Symbol.iterator` 为每一个对象定义了默认的迭代器。该迭代器可以被 `for...of` 循环使用。

Code block

```
1 const iterable1 = {};
2
3 iterable1[Symbol.iterator] = function* () {
4   yield 1;
5 }
```

`margin` 重叠问题指的是相邻元素之间的垂直 `margin` 值会合并成一个，而不是两个 `margin` 值简单相加。

1. 添加边框或内边距:

在元素之间添加边框或内边距可以阻止 `margin` 的合并，因为边框和内边距不会被重叠。

2. 使用 `overflow` 属性:

在父元素上设置 `overflow: hidden;` 或其他非 `visible` 值，可以阻止 `margin` 塌陷。

3. 使用Flexbox 或Grid 布局:

Flexbox 和Grid 布局在布局元素时，通常不会发生 `margin` 重叠。

4. 使用 `clear` 属性:

`clear` 属性主要用于清除浮动，但也可以用来阻止 `margin` 重叠，特别是在浮动元素和其他元素之间。

5. 调整 `margin` 值:

如果可能，可以调整元素的 `margin` 值，使其不会重叠。例如，如果两个元素之间需要20px 的间距，可以给其中一个元素设置 `margin-bottom: 20px;`，另一个元素设置 `margin-top: 0;`。

6. 使用负 `margin`:

在某些情况下，可以使用负 `margin` 来抵消重叠的 `margin`。

7. 使用BFC (Block Formatting Context):

创建BFC 也可以阻止 `margin` 重叠。例如，设置 `overflow: hidden;` 或 `display: flow-root;` 可以创建一个BFC。

2.9 箭头函数

1. 没有自己的 `this`

箭头函数没有自己的 `this` 绑定，它会捕获其所在上下文的 `this` 值。

```

5     yield 2;
6     yield 3;
7 };
8
9 console.log([...iterable1]);
10 // Expected output: Array [1, 2, 3]

```

内置可迭代对象

```

Code block
1  const arr = [1, 2, 3];
2  for (const item of arr) {
3      console.log(item); // 1, 2, 3
4  }
5
6  const str = "hello";
7  for (const char of str) {
8      console.log(char); // h, e, l, l, o
9  }
10
11 const map = new Map([[ 'a', 1 ], [ 'b', 2 ]]);
12 for (const [key, value] of map) {
13     console.log(key, value); // a 1, b 2
14 }

```

使自定义对象可迭代

```

Code block
1  const myIterable = {
2      data: [10, 20, 30],
3      [Symbol.iterator]() {
4          let index = 0;
5          return {
6              next: () => {
7                  if (index < this.data.length) {
8                      return { value: this.data[index++],
9                          done: false };
10                 }
11                 return { done: true };
12             }
13         };
14     };
15
16     for (const item of myIterable) {
17         console.log(item); // 10, 20, 30
18     }

```

1. 协议一致性：`Symbol.iterator` 方法必须返回一个迭代器对象，该对象包含一个 `next()` 方法。
2. `next()` 方法：每次调用 `next()` 返回一个包含两个属性的对象：
 - `value`：当前迭代的值
 - `done`：布尔值，表示迭代是否完成

特性	传统函数	箭头函数
this 绑定时机	调用时确定	定义时确定
this 可变性	可改变	不可改变
适用场景	需要动态 this 的场景	需要固定 this 的场景

2. 不适合作为方法函数

```

Code block
1  const obj = {
2      name: "Alice",
3      traditionalFunc: function() {
4          console.log(this.name); // Alice
5      },
6      arrowFunc: () => {
7          console.log(this.name); // undefined (或
            window.name 在浏览器中)
8      }
9  };
10
11 obj.traditionalFunc(); // 正确显示 "Alice"
12 obj.arrowFunc(); // 不显示或显示全局的 name

```

3. 继承外层作用域的 this

```

Code block
1  const obj = {
2      name: 'Alice',
3      sayName: function() {
4          // 传统函数, this 取决于调用方式
5          console.log(this.name);
6
7          setTimeout(function() {
8              console.log(this.name); // undefined
                (this 指向 window/global)
9          }, 100);
10
11          setTimeout(() => {
12              console.log(this.name); // 'Alice' (继承
                外层 this)
13          }, 100);
14      }
15  };
16
17 obj.sayName();

```

4. 无法通过 call/apply/bind 改变 this

```

Code block
1  const obj1 = { value: 1 };
2  const obj2 = { value: 2 };
3
4  const fn = () => console.log(this.value);
5
6  fn.call(obj1); // undefined (this 不会改变)
7  fn.apply(obj2); // undefined

```

3. 自动调用：在以下情况下会自动调用 `Symbol.iterator` 方法：

- `for...of` 循环
- 展开运算符 `...`
- 解构赋值
- `Array.from()`
- `new Set()`，`new Map()` 等构造函数

4. 可提前终止：迭代器可以可选地实现 `return()` 方法，用于在迭代提前退出时执行清理工作

生成器函数简化迭代器

```
Code block
1  const myIterable = {
2    *[Symbol.iterator]() {
3      yield 1;
4      yield 2;
5      yield 3;
6    }
7  };
8
9  console.log([...myIterable]); // [1, 2, 3]
10
```

检查对象是否可迭代

```
Code block
1  function isIterable(obj) {
2    return obj != null && typeof
    obj[Symbol.iterator] === 'function';
3  }
4
5  console.log(isIterable([])); // true
6  console.log(isIterable({})); // false
```

```
8  const boundFn = fn.bind(obj1);
9  boundFn(); // undefined
```

5. 不能作为构造函数

```
Code block
1  const Person = (name) => {
2    this.name = name; // TypeError: 箭头函数不能
    用作构造函数
3  };
4
5  // 正确的方式是使用传统函数
6  function Person(name) {
7    this.name = name;
8  }
```

6. 没有 arguments 对象

箭头函数没有自己的 `arguments` 对象，但可以访问外围函数的 `arguments`。

```
Code block
1  function outer() {
2    const inner = () => {
3      console.log(arguments); // 访问 outer 的
    arguments
4    };
5    inner();
6  }
7
8  outer(1, 2, 3); // 输出 [1, 2, 3]
```

7. 没有 prototype 属性

```
Code block
1  const arrow = () => {};
2  console.log(arrow.prototype); // undefined
```

8. 不能用作生成器函数

```
Code block
1  const gen = *() => {}; // SyntaxError
```

9. 适合回调函数

```
Code block
1  class Button {
2    constructor() {
3      this.text = 'Click me';
4      this.element =
    document.createElement('button');
5
6      // 传统函数需要 bind
7      this.element.addEventListener('click',
    function() {
```

```
8         console.log(this.text); // undefined
          (this 指向 DOM 元素)
9     });
10
11     // 箭头函数自动绑定
12     this.element.addEventListener('click', ()
=> {
13         console.log(this.text); // "Click me"
14     });
15 }
16 }
```

- 在需要固定 `this` 的场景使用箭头函数（如回调函数）
- 需要动态 `this` 时使用传统函数
- 对象方法使用传统函数或方法简写语法
- 类方法使用传统函数或类字段箭头函数（需注意内存占用）

2.10 Map

- 键可以是任意数据类型（对象、函数等）
- 保持插入顺序
- 高效的查找操作（接近 O(1) 时间复杂度）
- 可以直接遍历

Code block

```
1  const map = new Map();
2
3  // 添加元素
4  map.set('name', 'Alice');
5  map.set(1, 'number one');
6
7  // 获取元素
8  map.get('name'); // 'Alice'
9
10 // 检查键是否存在
11 map.has('name'); // true
12
13 // 删除元素
14 map.delete('name');
15
16 // 大小
17 map.size;
18
19 // 遍历
20 map.forEach((value, key) => console.log(key,
value));
21 for (const [key, value] of map) { /* ... */ }
```

使用场景

1. 需要键不是字符串的情况

Code block

```
1  const domNodes = new Map();
2  const element =
document.querySelector('#someId');
```

2.11 WeakMap

1. 键必须是对象（不能是原始值）
2. 键是弱引用的（不会阻止垃圾回收）
3. 不可枚举（没有方法可以获取所有键或值）
4. 没有 size 属性

vue3 proxy 使用weakMap，用于处理原始对象不存在时

1. 自动内存管理：当组件销毁时，相关响应式数据自动解除关联
2. 无额外清理负担：开发者无需手动清理响应式引用

Code block

```
1  const weakMap = new WeakMap();
2
3  const obj1 = {};
4  const obj2 = {};
5
6  // 设置键值对
7  weakMap.set(obj1, 'private data 1');
8  weakMap.set(obj2, 'private data 2');
9
10 // 获取值
11 console.log(weakMap.get(obj1)); // 'private
data 1'
12
13 // 检查键是否存在
14 console.log(weakMap.has(obj1)); // true
15
16 // 删除键值对
17 weakMap.delete(obj1);
```

1. 弱引用特性

WeakMap 对键是弱引用，这意味着：

- 如果没有其他引用指向键对象，该对象会被垃圾回收
- 即使 WeakMap 中仍然引用该对象，也不会阻止垃圾回收

```
3 domNodes.set(element, { clicks: 0 });
```

2. 需要维护插入顺序的键值对

Code block

```
1 const orderedData = new Map();
2 orderedData.set('first', 1);
3 orderedData.set('second', 2);
```

3. 频繁增删键值对的场景

Code block

```
1 const cache = new Map();
2 function getData(key) {
3   if (cache.has(key)) return cache.get(key);
4   const data = fetchData(key); // 假设的获取数据函数
5   cache.set(key, data);
6   return data;
7 }
```

```
1 let obj = { id: 1 };
2 const weakMap = new WeakMap();
3 weakMap.set(obj, 'some data');
4
5 obj = null; // 现在 { id: 1 } 可以被垃圾回收,
              WeakMap 中的条目会自动移除
```

2. 不可迭代

WeakMap 没有以下方法:

- keys() values() entries() forEach() size 属性

3. 键必须是对象

Code block

```
1 const weakMap = new WeakMap();
2
3 weakMap.set({}, 'valid'); // 正确
4 weakMap.set('key', 'value'); // TypeError:
    Invalid value used as weak map key
```

使用场景

1. 缓存计算结果 当对象不存在时自动清除

Code block

```
1 const cache = new WeakMap();
2
3 function computeExpensiveValue(obj) {
4   if (cache.has(obj)) {
5     return cache.get(obj);
6   }
7
8   const result = /* 复杂计算 */;
9   cache.set(obj, result);
10  return result;
11 }
```

2. 存储 DOM 元素的附加数据 (元素移除时自动清理)

Code block

```
1 const elementHandlers = new WeakMap();
2
3 function setupHandler(element) {
4   const handler = () =>
5     console.log('Clicked!');
6   element.addEventListener('click', handler);
7   elementHandlers.set(element, handler);
8 }
9
10 function teardownHandler(element) {
11   const handler =
12     elementHandlers.get(element);
13   element.removeEventListener('click', handler);
14   elementHandlers.delete(element);
15 }
```


2.12 Set

Code block

```
1  const uniqueNames = new Set();
2  uniqueNames.add('Alice'); // 字符串
3  uniqueNames.add(42);      // 数字
4  uniqueNames.add({});      // 对象
```

需要遍历集合或知道集合大小

Code block

```
1  const colors = new Set(['red', 'green',
2    'blue']);
3  console.log(colors.size); // 3
4  for (const color of colors) {
5    console.log(color);
6  }
```

需要实现集合运算（并集、交集等）

Code block

```
1  const setA = new Set([1, 2, 3]);
2  const setB = new Set([2, 3, 4]);
3
4  // 并集
5  const union = new Set([...setA, ...setB]);
6
7  // 交集
8  const intersection = new
  Set([...setA].filter(x => setB.has(x)));
```

2.14 Flex 属性

`flex` 属性的完整写法是 `flex: <flex-grow> <flex-shrink> <flex-basis>`。

- `flex-grow`：
 - 定义项目的放大比例，默认为0。如果所有项目的 `flex-grow` 都大于0，它们将等比例地放大以占据剩余空间。
- `flex-shrink`：
 - 定义项目的缩小比例，默认为1。如果所有项目的 `flex-shrink` 都大于0，它们将等比例地缩小以适应容器空间。
- `flex-basis`：
 - 定义了分配多余空间之前，项目占据的主轴空间大小，默认为 `auto`。可以设置具体数值(如50px, 20%) 或使用 `auto`。

常用的 `flex` 属性值:

- `flex: 1` 或 `flex: 1 1 0%`:元素会占据所有可用的剩余空间，并且可以根据需要缩小。

2.13 WeakSet

核心区别对比

特性	Set	WeakSet
值类型	任意类型（对象、原始值等）	只能是对象（非原始值）
垃圾回收	强引用（阻止值被回收）	弱引用（不阻止值被回收）
可迭代性	可遍历（keys/values/entries）	不可遍历
大小查询	有 <code>size</code> 属性	无 <code>size</code> 属性
方法完整性	完整API（add/delete/has等）	只有基本操作（add/delete/has）
内存管理	可能导致内存泄漏	自动清理无引用的值

2.15 Grid 属性

CSS Grid 布局属性主要用于创建基于行和列的二维网格布局。容器元素通过 `display: grid` 声明为网格

- `grid-template-columns`:定义网格的列数和每列的宽度。
 - 例如，`grid-template-columns: 100px 200px auto;` 定义了三列，第一列宽100px，第二列宽200px，第三列自动填充剩余空间。
- `grid-template-rows`:定义网格的行数和每行的高度。
- 用法与 `grid-template-columns` 类似。
- `grid-template-areas`:定义命名的网格区域，并将网格项目放置到这些区域中。需要配合 `grid-area` 属性使用，用于将元素放置到指定的网格区域中。

隐式网格属性（控制自动生成的行或列）：

- `grid-auto-columns`:当网格项目超出显式定义的列时，定义自动生成的列的尺寸。

- `flex: auto` 或 `flex: 1 1 auto`:元素会根据内容调整大小, 并且可以根据需要放大或缩小。
- `flex: none` 或 `flex: 0 0 auto`:元素不会伸缩, 保持其原始大小。
- `flex: initial` 或 `flex: 0 1 auto`:元素的 `flex-grow` 为0, `flex-shrink` 为1, `flex-basis` 为 `auto`。

- `grid-auto-rows`:当网格项目超出显式定义的行时, 定义自动生成的行的尺寸。
- `grid-auto-flow`:定义自动放置项目的方向和顺序。

其他重要属性:

- `grid-gap`:设置行和列之间的间隙 (网格间距)。
- `grid-column` / `grid-row`:用于在网格容器内定位网格项目, 指定项目所在的列和行范围。
- `grid-area`:用于指定网格项目所在的区域名称, 需要与 `grid-template-areas` 配合使用。

2.16 水平垂直居中

Code block

```
1 // 绝对定位 +
  translate
2 .parent {
3   position:
  relative;
4 }
5
6 .child {
7   position:
  absolute;
8   top: 50%;
9   left: 50%;
10  transform:
  translate(-50%,
    -50%);
11 }
```

Code block

```
1 // flex
2 .parent {
3   display: flex;
4   justify-
    content:
  center; /* 水平居
    中 */
5   align-items:
  center; /*
    垂直居中 */
6 }
```

Code block

```
1 // grid
2 .parent {
3   display: grid;
4   place-items:
  center; /* 水平垂
    直居中 */
5 }
```

Code block

```
1 // table cell
2 .parent {
3   display:
  table-cell;
4   text-align:
  center; /*
    水平居中 */
5   vertical-
    align: middle;
  /* 垂直居中 */
6 }
7
8 .child {
9   display:
  inline-block;
10 }
```

2.17 数组去重

Code block

```
1 // 1. 使用 Set
2 const uniqueArray = [...new Set(array)];
3 // 2. filter
4 const uniqueArray = array.filter((item, index) => array.indexOf(item) === index);
5 // 3. reduce
6 const uniqueArray = array.reduce((acc, current) => {
7   if (!acc.includes(current)) {
8     acc.push(current);
9   }
10  return acc;
11 }, []);
12
13 // 4. 数组遍历去重
14 const uniqueArray = [];
15 for (let i = 0; i < array.length; i++) {
16   if (uniqueArray.indexOf(array[i]) === -1) {
17     uniqueArray.push(array[i]);
18   }
19 }
```

2.18 防抖

2.19 节流

在事件被触发n秒后再执行回调，如果在这n秒内又被触发，则重新计时

Code block

```
1 function debounce(func, delay) {
2   let timeoutId;
3   return function(...args) {
4     clearTimeout(timeoutId);
5     timeoutId = setTimeout(() => {
6       func.apply(this, args);
7     }, delay);
8   };
9 }
```

Code block

```
1 import { useCallback, useEffect, useRef } from
  'react';
2
3 function useDebounce(callback, delay) {
4   const timeoutRef = useRef(null);
5
6   const debouncedCallback =
  useCallback((...args) => {
7     if (timeoutRef.current) {
8       clearTimeout(timeoutRef.current);
9     }
10
11     timeoutRef.current = setTimeout(() => {
12       callback(...args);
13     }, delay);
14   }, [callback, delay]);
15
16   useEffect(() => {
17     return () => {
18       if (timeoutRef.current) {
19         clearTimeout(timeoutRef.current);
20       }
21     };
22   }, []);
23
24   return debouncedCallback;
25 }
```

规定在一个单位时间内，只能触发一次函数。如果这个单位时间内触发多次函数，只有一次生效

Code block

```
1 function throttle(func, limit) {
2   let lastFunc;
3   let lastRan;
4   return function(...args) {
5     const context = this;
6     if (!lastRan) {
7       func.apply(context, args);
8       lastRan = Date.now();
9     } else {
10      clearTimeout(lastFunc);
11      lastFunc = setTimeout(() => {
12        if ((Date.now() - lastRan) >= limit) {
13          func.apply(context, args);
14          lastRan = Date.now();
15        }
16      }, limit - (Date.now() - lastRan));
17    }
18  };
19 }
```

Code block

```
1 import { useCallback, useRef } from 'react';
2
3 function useThrottle(callback, delay) {
4   const lastExecuted = useRef(Date.now());
5   const timeoutRef = useRef(null);
6
7   const throttledCallback =
  useCallback((...args) => {
8     const now = Date.now();
9     const timeSinceLastExecution = now -
    lastExecuted.current;
10    const delayRemaining = delay -
    timeSinceLastExecution;
11
12    if (delayRemaining <= 0) {
13      // 足够时间过去了，可以立即执行
14      lastExecuted.current = now;
15      callback(...args);
16    } else if (!timeoutRef.current) {
17      // 设置定时器在剩余时间后执行
18      timeoutRef.current = setTimeout(() => {
19        lastExecuted.current = Date.now();
20        callback(...args);
21        timeoutRef.current = null;
22      }, delayRemaining);
23    }
24  }, [callback, delay]);
25
26  // 清理定时器
27  useEffect(() => {
28    return () => {
29      if (timeoutRef.current) {
30        clearTimeout(timeoutRef.current);
31      }
32    }
33  });
34 }
```

```
32     };
33     }, []);
34
35     return throttledCallback;
36 }
```

2.20 CSS 两栏

1. 双float方案

利用CSS的 `calc()` 函数来计算右侧宽度值, 需要清除父容器的浮动

```
Code block
1  .container::after {
2      content: "";
3      display: block;
4      clear: both;
5  }
6
7  .left, .right {
8      box-sizing: border-box;
9      float: left;
10 }
11
12 .right {
13     width: calc(100% - 120px);
14 }
```

2. float + margin-left

`block` 的元素盒子的宽度具有填满父容器, 并随着父容器的宽度自适应的流动特性。并且 block元素和浮动元素并排时, block元素会忽视浮动元素, 给左侧盒子浮动, 右侧设置margin-left保持距离即可

```
Code block
1  .container::after {
2      content: "";
3      display: block;
4      clear: both;
5  }
6
7  .left {
8      box-sizing: border-box;
9      float: left;
10 }
11
12 .right {
13     margin-left: 120px;
14 }
```

3. absolute + margin-left

```
Code block
1  .left {
2      box-sizing: border-box;
3      position: absolute;
```

2.21 CSS 三栏

1. float+cac

```
Code block
1  div {
2      height: 100px;
3  }
4  .left {
5      float: left;
6      width: 200px;
7      background-color: red;
8  }
9  .middle {
10     float: left;
11     width: calc(100% - 400px);
12     background-color: green;
13 }
14 .right {
15     float: right;
16     width: 200px;
17     background-color: blue;
18 }
```

2. 浮动float + margin 负值

```
Code block
1  div {
2      height: 100px;
3  }
4  .left {
5      float: left;
6      width: 200px;
7      background-color: red;
8  }
9  .right {
10     float: right;
11     width: 200px;
12     background-color: blue;
13 }
14 .middle {
15     margin-left: 200px;
16     margin-right: 200px;
17     background-color: green;
18 }
```

3. 圣杯布局

利用**浮动**, **外边距负值**和**相对定位**来实现。

- 父级元素设置左右的 `padding` , 三列均设置向左浮动;


```

4   }
5
6   .right {
7       margin-left: 120px;
8   }

```

4. float + BFC 方法

Code block

```

1   .container::after {
2       content: "";
3       display: block;
4       clear: both;
5   }
6
7   .left {
8       float: left;
9   }
10
11  .right {
12      margin-left: 0;
13      overflow: auto;
14  }
15

```

5. Flex

Code block

```

1   .container {
2       display: flex;
3   }
4
5   .right {
6       flex: 1
7   }

```

- 中间一列放在最前面，宽度设置为父级元素的宽度，因此后面两列都被挤到了下一行；
- 再通过设置 `margin` 负值将其移动到上一行；
- 再利用相对定位，定位到两边。

Code block

```

1   .container {
2       height: 100px;
3       padding: 0 200px 0 200px;
4       overflow: hidden;
5   }
6   .middle,
7   .left,
8   .right {
9       float: left;
10      height: 100%;
11  }
12  .middle {
13      width: 100%;
14      background: green;
15  }
16  .left {
17      position: relative;
18      width: 200px;
19      margin-left: -100%;
20      left: -200px;
21      background: red;
22  }
23  .right {
24      position: relative;
25      width: 200px;
26      margin-right: -200px;
27      background: blue;
28  }

```

4. 双飞翼布局

左右位置的保留是通过中间列的 `margin` 值来实现的，而不是通过父元素的 `padding` 来实现的。

Code block

```

1   .container {
2       height: 100px;
3       overflow: hidden;
4   }
5   .middle,
6   .left,
7   .right {
8       float: left;
9       height: 100%;
10  }
11  .middle {
12      width: 100%;
13      background-color: green;
14  }
15  .inner {
16      height: 100%;
17      margin: 0 200px 0 200px;

```

```

18  }
19  .left {
20      width: 200px;
21      margin-left: -100%;
22      background-color: red;
23  }
24  .right {
25      width: 200px;
26      margin-left: -200px;
27      background-color: blue;
28  }

```

5. Flex

Code block

```

1  div {
2      height: 100px;
3  }
4  .container {
5      display: flex;
6  }
7  .left {
8      width: 200px;
9      background: red;
10 }
11 .middle {
12     flex: 1;
13     background: green;
14 }
15 .right {
16     background: blue;
17     width: 200px;
18 }
19

```

6. Grid

Code block

```

1  div {
2      height: 100px;
3  }
4  .container {
5      display: grid;
6      /* 核心代码：左右两栏固定宽度，中间自适应宽度 */
7      grid-template-columns: 200px auto 200px;
8  }
9  .left {
10     background: red;
11 }
12 .middle {
13     background: green;
14 }
15 .right {
16     background: blue;
17 }

```

2.22 瀑布流

Grid

- 1. 定义网格容器: 设置 `display: grid;` 来启用grid 布局。
- 2. 设置列数和宽度: 使用 `grid-template-columns` 属性来定义列数和每列的宽度。例如, `grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));` 会根据容器宽度自动调整列数, 每列最小宽度为250px, 最大宽度为1fr (剩余空间平分)。
- 3. 设置行高: 使用 `grid-auto-rows` 属性来设置每行的高度, 例如 `grid-auto-rows: 10px;`。
- 4. 设置间距: 使用 `gap` 属性来设置列和行之间的间距, 例如 `gap: 10px;`。
- 5. 控制元素跨行: 使用 `grid-row` 属性来控制元素跨越的行数, 从而模拟不同高度的元素。

flex

- 1. 定义容器: 设置 `display: flex;` 来启用flex 布局。
- 2. 设置列数: 使用 `flex-wrap: wrap;` 属性使元素可以换行, 配合 `width` 或 `flex-basis` 属性来控制每列的宽度。
- 3. 设置间距: 使用 `margin` 属性来控制元素之间的间距。
- 4. 模拟瀑布流效果: 可以通过JS 动态地改变每个元素的高度, 使其看起来像瀑布流。

column-count

Code block

```
1 <div class="waterfall">
2   <div class="item">...
3   </div>
4   <div class="item">...
5   </div>
6   <!-- 更多item -->
7 </div>
8 <style>
9   .waterfall {
10     column-count: 4;
11     /* 列数 */
12     column-gap: 15px;
13     /* 列间距 */
14   }
15   .item {
16     break-inside: avoid;
17     /* 防止元素被分割到不同列 */
18     margin-bottom: 15px;
19     /* 行间距 */
20   }
21 </style>
```

其他库

如 [Masonry](#) 或 [Isotope](#)

绝对定位 + JavaScript

- 窗口resize时重新计算
- 图片使用 `imagesLoaded` 库确保高度计算准确

Code block

```
1 function waterfall(container, items, cols) {
2   const containerWidth = container.offsetWidth;
3   const colWidth = containerWidth / cols;
4   const colHeights = new Array(cols).fill(0);
5
6   items.forEach(item => {
7     const minHeight = Math.min(...colHeights);
8     const colIndex = colHeights.indexOf(minHeight);
9
10    item.style.position = 'absolute';
11    item.style.width = `${colWidth}px`;
12    item.style.left = `${colIndex * colWidth}px`;
13    item.style.top = `${minHeight}px`;
14
15    colHeights[colIndex] += item.offsetHeight;
```

性能优化相关

- 1. IntersectionObserver 懒加载
- 2. 虚拟滚动: 只渲染可视区域内的元素
- 3. 防抖resize: 优化窗口大小改变时的重排
- 4. CSS硬件加速

Code block

```
1 .item {
2   transform: translateZ(0);
3 }
```

- 5. 渐进式加载 先加载低质量图片占位

```
16     });
17
18     container.style.height =
19       `${Math.max(...colHeights)}px`;
20   }
```

2.23 CSS 硬件加速

CSS 硬件加速（也称为 GPU 加速）是通过利用计算机的图形处理单元（GPU）而不是中央处理器（CPU）来渲染页面元素的技术，可以显著提升动画和渲染性能。

当浏览器检测到某些 CSS 属性被应用时，它会将这些元素提升到它们自己的"层"(layer)，然后由 GPU 来合成这些层，而不是由 CPU 来重绘整个页面

3D 变换属性

Code block

```
1 .element {
2   transform:
3     translate3d(0, 0, 0);
4   /* 或者 */
5   transform:
6     translateZ(0);
7 }
```

will-change

Code block

```
1 .element {
2   will-change: transform;
3   /* 提前告知浏览器元素可能的变
4      化 */
5 }
```

perspective

Code block

```
1 .element {perspective:
2   1000px;}
```

- 1. 更流畅的动画：GPU 专为图形计算优化 | 减少重绘：独立的合成层减少页面重绘
- 2. 降低CPU负载：将图形工作转移给GPU | 移动端性能提升：特别有利于移动设备

2.24 响应式实现

- 1. 基于媒体查询
- 2. Rem
- 3. vh/vw

原理

窗口设置（viewport）

- width=device-width：让页面宽度等于设备宽度
- initial-scale=1.0：初始缩放比例为1:1
- maximum-scale=1.0：禁止用户缩放(谨慎使用)
- user-scalable=no：禁止用户缩放(不推荐)

媒体查询

Code block

```
1 /* 手机设备 (小于768px) */
2 @media (max-width: 767px) {
3   .container { padding: 0 10px; }
4 }
5
6 /* 平板设备 (768px到991px) */
7 @media (min-width: 768px) and (max-width:
8   991px) {
9   .container { width: 750px; }
10 }
```

2.25 图片响应式

1. max-width

Code block

```
1 img {
2   display: inline-block;
3   max-width: 100%;
4   height: auto;
5 }
6
```

2. srcset

dpi = 1的话则加载1倍图，而dpi = 2则加载2倍图

Code block

```
1 
```

```

11  /* 桌面设备 (大于992px) */
12  @media (min-width: 992px) {
13    .container { width: 970px; }
14  }

```

2.26 主题切换

Css var

- 纯CSS实现，性能好
- 支持实时切换无闪烁

Code block

```

1  :root {
2    --primary-color: #4285f4;
3    --secondary-color: #34a853;
4    --text-color: #202124;
5    --bg-color: #ffffff;
6  }
7
8  [data-theme="dark"] {
9    --primary-color: #8ab4f8;
10   --secondary-color: #81c995;
11   --text-color: #e8eae8;
12   --bg-color: #202124;
13 }
14
15 // js切换
16 function toggleTheme() {
17   const currentTheme =
18     html.getAttribute('data-theme');
19   html.setAttribute('data-theme', newTheme);
20   localStorage.setItem('theme', newTheme);

```

动态加载主题

- 主题完全隔离
- 可按需加载减少初始包体积

Code block

```

1  function loadTheme(themeName) {
2    const link = document.createElement('link');
3    link.rel = 'stylesheet';
4    link.href = `/themes/${themeName}.css`;
5    link.id = 'theme-style';
6
7    const oldLink =
8      document.getElementById('theme-style');
9    if (oldLink) {
10     document.head.replaceChild(link, oldLink);
11   } else {
12     document.head.appendChild(link);
13   }

```

CSS 预处理器

- 编译时确定主题，性能最优
- 适合主题数量固定的场景

Code block

```

1  $themes: (
2    light: (
3      primary: #4285f4,
4      bg: #ffffff,
5      text: #202124
6    ),
7    dark: (
8      primary: #8ab4f8,
9      bg: #202124,
10     text: #e8eae8
11   )
12 );
13
14 @mixin theme($property, $key) {
15   @each $theme-name, $theme-map in $themes {
16     [data-theme="#{$theme-name}"] & {
17       #{$property}: map-get($theme-map, $key);
18     }
19   }
20 }

```

CSS-in-JS、tailwind

Code block

```

1  import { ThemeProvider } from 'styled-
2    components';
3
4  const lightTheme = {
5    colors: {
6      primary: '#4285f4',
7      bg: '#ffffff',
8      text: '#202124'
9    }
10 };
11
12 const darkTheme = {
13   colors: {
14     primary: '#8ab4f8',
15     bg: '#202124',
16     text: '#e8eae8'
17   }

```


3. Vue

3.1 Vue2 Vue3双向绑定 对比

Vue 2 通过 `Object.defineProperty`

- 监听不到对象属性的删除和添加，需要借助 `$set` 和 `$delete` 方法。
- 对于数组的API方法无法监听，vue2中对于数组的方法进行了包裹。
- 数据变化时通知依赖的Watcher 更新视图，需要递归遍历对象来设置getter 和setter，浪费性能。

Vue 3 采用Proxy 监听整个对象，无需递归

3.2 Vue3 比 Vue2 多了什么

- 性能提升:
 - **优化虚拟DOM (Virtual DOM)** Vue 3 对虚拟DOM进行了优化，例如静态节点提升、静态props优化等，减少了不必要的DOM操作，提高了渲染性能
 - Vue 3 Proxy -> Vue 2 的Object.defineProperty
 - **碎片(Fragments)** Vue 3 支持碎片，允许组件返回多个根节点，也减少了不必要的DOM层级，优化了渲染性能
 - Vue 3 支持tree shaking
 - vue 3 静态标记基础上，支持最长递增子序列
- 响应式系统:
 - Vue 3 Proxy -> Vue 2 的Object.defineProperty
- 代码组织:
 - Vue 3 引入了Composition API，允许开发者更好地组织和复用组件逻辑，使得代码更易于维护和测试。
 - 源码层面上优化 Vue 3 采用模块化设计，将各个组件功能分离，增强了应用程序的灵活性和可扩展性。
- TypeScript 支持:
 - Vue 3 原生支持TypeScript

3.4 Ref reactive 区别

3.3 vue生命周期

1. 创建(Creation):
 - `beforeCreate` : 在实例被创建之初，数据观测(data observer) 和事件(event/watcher) 都尚未初始化。
 - `created` : 实例已经创建完成，数据观测(data observer)、属性和方法的运算，以及watch/event 事件回调都已设置完成，但还没有挂载到DOM 上。
2. 挂载(Mounting):
 - `beforeMount` : 在模板编译/渲染成DOM 之前被调用。
 - `mounted` : 实例已经挂载到DOM 上，此时可以访问真实的DOM 节点。
3. 更新(Updating):
 - `beforeUpdate` : 在数据发生变化，DOM 重新渲染之前被调用。
 - `updated` : 组件DOM 已经更新完成。
4. 销毁(Destruction):
 - `beforeDestroy` : 实例销毁之前调用。在这一步，实例仍然完全可用。
 - `destroyed` : 实例销毁后调用。调用后，Vue 实例的所有指令都解除绑定，所有事件监听器都被移除，所有的子实例也都会被销毁

vue3 新增 `renderTriggered` `renderTracked`

Code block

```
1 // 简化的 reactive 实现
2 function reactive(target) {
3   return new Proxy(target, {
4     get(target, key, receiver) {
5       track(target, key); // 依赖收集
6       return Reflect.get(target, key,
7         receiver);
8     },
9     set(target, key, value, receiver) {
```

特性	ref	reactive
适用数据类型	基本类型和对象	只适用于对象（包括数组）
访问方式	需要通过 <code>.value</code> 访问	直接访问属性
响应式原理	使用对象包装 + getter/setter	使用 Proxy 代理
模板自动解包	自动解包（无需 <code>.value</code> ）	直接使用
重新赋值能力	可以整个替换	不能替换整个对象
TypeScript 支持	更好的类型推断	嵌套属性类型推断较复杂

Code block

```
1 // 简化的 ref 实现
2 function ref(value) {
3   const refObject = {
4     get value() {
5       track(refObject, 'value'); // 依赖收集
6       return value;
7     },
8     set value(newVal) {
9       value = newVal;
10      trigger(refObject, 'value'); // 触发更新
11    }
12  };
13   return refObject;
14 }
```

3.5 Vue nextTick 实现

在 DOM 更新周期后执行延迟回调，主要基于 JavaScript 的微任务队列机制

Vue 3 使用 Promise 作为 `nextTick` 的实现基础，原因包括1：

1. 微任务优先级高于宏任务（如 `setTimeout`），能更早执行
2. 现代浏览器广泛支持 Promise
3. 可以形成良好的 Promise 链式调用

Code block

```
1 const resolvedPromise = Promise.resolve() as
  Promise<any>
2 let currentFlushPromise: Promise<void> | null
  = null
3
4 export function nextTick<T = void>(<
5   this: T,
6   fn?: (this: T) => void
7   ): Promise<void> {
8   const p = currentFlushPromise ||
    resolvedPromise
9   return fn ? p.then(this ? fn.bind(this) :
    fn) : p
10 }
```

3.6 Vue 通信

1. 父子组件通信

```
9 Reflect.set(target, key, value,
    receiver);
10 trigger(target, key); // 触发更新
11 return true;
12 }
13 });
14 }
```

1. Ref -> reative: reactive

2. Reative ->ref: toRefs

使用建议

- 对于简单值，`ref` 比 `reactive` 更轻量
- 对于复杂对象，`reactive` 比深度 `ref` 更高效
- 在模板中使用大量 `ref` 会导致更多 `.value` 访问（虽然模板会自动解包）

Vue 2 中 `nextTick` 的实现更加复杂，有一个降级策略：

1. 优先使用 Promise
2. 不支持 Promise 时回退到 MutationObserver
3. 再不支持则使用 `setImmediate`
4. 最后使用 `setTimeout`

2. 兄弟组件通信

- **Props**: 父组件通过 `props` 向子组件传递数据。子组件通过 `props` 接收数据。
- `$emit` 和事件 `$on`
- `$parent` 和 `$children`: `$parent` 访问父组件实例, `$children` 访问所有子组件实例。不推荐直接访问组件实例, 因为这会破坏组件的封装性。
- `provide` 和 `inject`
- `$attrs` 和 `$listeners`:

- `eventBus` :创建一个空的Vue 实例作为事件总线, 通过 `bus.$emit` 触发事件, `bus.$on` 监听事件。这种方式简单, 但适用于小型应用, 对于大型应用, 更推荐使用
- `Vuex` :Vuex 是一个专为Vue.js 应用程序开发的状态管理模式。它采用集中式存储管理应用的所有组件的状态, 并以相应的规则保证状态的唯一性。

3. 跨层级组件通信

- `eventBus` :同上, 适用于小型应用。
- `Vuex` :同上, 适用于大型应用。
- `provide` 和 `inject` :同上。

3.7 Vue 状态管理

方案	适用场景	优点	缺点	复杂度	适用规模
组件内状态 (data)	单个组件内部状态	简单直接, 无需额外依赖	难以跨组件共享	低	小型应用
Props/Events	父子组件通信	Vue 原生支持, 简单明了	多层嵌套时繁琐	低	小型应用
Provide/Inject	深层嵌套组件通信	避免 prop 逐层传递	非响应式(默认), 组织性较差	中	中小型应用
Event Bus	任意组件间通信	简单全局通信方案	难以追踪调试, 可能内存泄漏	中	不推荐使用
Vuex	中大型应用集中式状态	集中管理, 调试工具支持, 可预测状态变化	相对繁琐, 学习曲线	高	中大型应用
Pinia	Vue 2/3 应用状态管理	更简单的 API, TypeScript 支持, 模块化	较新, 生态较小	中高	各种规模
Composition API (reactive/ref)	组合式逻辑复用	灵活, 逻辑组合方便	需要良好组织	中	各种规模

4. React

4.1 Hooks 原理

1. **闭包(Closure)**:使得函数组件在每次渲染之间能够记住和访问之前的状态。
2. **Fiber 架构**: Fiber 节点上会保存一个链表, 用于存储组件中使用的Hooks。这个链表中的每个元素都对应一个Hook, 例如 `useState`, `useEffect` 等。

Fiber

4.2 useState 是同步还是异步

1. `useState` 和 `setState` 看起来是异步的, 但在实际执行上是同步的, 但会进行批处理优化, 以避免频繁的DOM 更新。
2. 在React 合成事件和生命周期钩子函数 (例如 `onClick`、`useEffect`) 中表现为“异步”,

```
1function FiberNode(  
2  tag: WorkTag,  
3  pendingProps: mixed,  
4  key: null | string,  
5  mode: TypeOfMode,  
6 ) {  
7   // Instance, 静态节点的数据结构属性  
8   this.tag = tag;  
9   this.key = key;  
10  this.elementType = null;  
11  this.type = null;  
12  this.stateNode = null;  
13  
14  // Fiber, 用来链接其他fiber节点形成的fiber树  
15  this.return = null;  
16  this.child = null;  
17  this.sibling = null;  
18  this.index = 0;  
19  
20  this.ref = null;  
21  
22  // 作为动态的工作单元的属性  
23  this.pendingProps = pendingProps;  
24  this.memoizedProps = null;  
25  this.updateQueue = null;  
26  this.memoizedState = null;  
27  this.dependencies = null;  
28 }
```

3. 在原生事件和 `setTimeout`、`Promise` 中的 `then`` 等回调函数中表现为同步

4.3 Vue React diff 复杂度

都是O(n)的时间空间复杂度

4.4 React 状态管理

状态管理库	核心特点	优点	缺点	适用场景
Redux	Flux 架构，单向数据流，不可变状态	- 可预测的状态管理 - 强大的调试工具（Redux DevTools） - 丰富的中间件生态（如 Redux Thunk/Saga）	- 模板代码多，配置繁琐 - 学习成本较高 - 性能优化需手动处理	大型复杂应用，需要严格状态控制的项目
Zustand	轻量级，基于 Hook，类似 Pinia 设计	- 极简 API，无需 Provider - 高性能，按需更新 - 支持中间件（如 Immer）	- DevTools 需额外配置 - 社区生态较新	中小型项目，快速开发，替代 Redux 的场景
MobX	响应式编程，可变状态	- 代码简洁，自动依赖追踪 - 适合 OOP 风格开发	- 调试困难（可变状态） - 异步更新可能导致意外行为	需要灵活状态管理的项目，适合熟悉响应式编程的团队
Context API	React 原生方案，基于 Provider/Consumer	- 无需额外依赖 - 适合简单状态共享	- 性能问题（嵌套 Provider 导致全量渲染） - 不适合复杂状态逻辑	小型应用，解决 Prop Drilling 问题

Jotai	原子化状态管理，类似 Recoil	- 极简 API，按需更新 - 适合细粒度状态控制 - 组合性强	- 依赖 React 上下文，无法在非 React 环境使用	表单、编辑器等需要局部状态管理的场景
Recoil	Facebook 出品，原子化状态	- 精准订阅，避免冗余渲染 - 支持派生状态（Selector）	- 文档较少，社区支持较弱 - 状态依赖链复杂时难维护	实验性项目，需要精细状态控制的场景
Valtio	基于 Proxy 的响应式状态	- 类 Vue 的响应式体验 - 代码量极少	- 可变状态调试困难 - 与 React 设计哲学冲突	快速原型开发，适合熟悉 Vue 的开发者

补充说明：

1. **性能对比**：Zustand 和 Jotai 在中小型项目中性能表现优异，而 Redux 在大型应用中更稳定。
2. **开发体验**：Zustand 和 Valtio 的 API 最简洁，适合快速迭代；Redux 适合需要严格架构约束的团队。
3. **异步处理**：Redux + Redux Thunk/Saga 最成熟，Zustand 和 MobX 也支持但更灵活。

4.5 React 19新特性

新特性

1. Actions 用了异步过渡的函数（该函数涉及到异步数据变更）被称作 “Actions”
2. useTransition

```
const [isPending, startTransition] = useTransition()
```
3. useFormStatus

```
const { pending } = useFormStatus();
```
4. useOptimistic

```
const [optimisticName, setOptimisticName] = useOptimistic(currentName);
```
5. use
 - a. 可以异步获取Promise 数据
 - b. 可以替换 useContext

use 与 Hook 的区别：

- **共同点**： use 和 Hook 一样，只能在渲染阶段调用。
 - **区别**： use 可以有条件地调用，而 Hook 必须在组件的顶层无条件调用。
6. React Server Components（服务器组件）

5. 其他

5.1 后端传大数据如何减少渲染负担

1. 数据分页与懒加载
 - 分页处理：只传输当前页面需要的数据

改进

1. 不再需要 forwardRef 就可以直接访问 ref 了
2. <Context> 代替 <Context.Provider>
3. useDeferredValue支持 initialValue
4. 新增对样式表 (stylesheets) 的原生支持，precedence 提前加载
5. 错误提示友好
 - onCaughtError**： 可以用来记录被错误边界捕获的错误，例如发送错误报告到监控系统。
 - onUncaughtError**： 可以用来处理未被捕获的错误，例如显示一个全局的错误提示，或者执行一些清理操作。
 - onRecoverableError**： 可以用来记录被自动恢复的错误，例如记录 Hydration 失败的情况

5.2 Infer

Infer用于条件类型 只能在条件类型的 extends 子句中使用

Code block

- 无限滚动：动态加载数据，用户滚动时再请求新数据
- intersection observer**
- 虚拟滚动：只渲染可视区域内的数据行 **scroll top**

2. 数据优化

- 压缩数据：使用Gzip等压缩算法减小传输体积

3. 前端渲染优化

- Web Workers：将数据处理移出主线程
- 分批渲染：将大数据分成小块逐步渲染

4. 缓存策略

- 本地存储：将不变的数据存储在localStorage或IndexedDB
- 服务端缓存：对频繁请求的数据进行缓存

5. 服务端优化

- 流式传输：使用SSE(Server-Sent Events)或WebSocket逐步发送数据
- GraphQL：让前端精确请求所需数据，避免过度获取

```
1  type UnpackArray<T> = T extends Array<infer U> ? U : T;
```

获取返回类型

```
Code block

1  type ReturnType<T> = T extends (...args: any[]) => infer R ? R : never;
2
3  function foo(): string {
4      return "hello";
5  }
6
7  type FooReturn = ReturnType<typeof foo>; // string
```

5.3 从输入url到页面渲染流程

1. DNS 解析

浏览器需要将域名（如 `www.example.com`）转换为服务器的 IP 地址。这个过程称为 ****DNS 解析****：

- a. 浏览器首先检查本地缓存，看是否已经缓存了该域名的 IP 地址。
- b. 如果本地缓存中没有，浏览器会向 **ISP（互联网服务提供商）的 DNS 服务器** 发送 DNS 查询请求。
- c. 如果 ISP 的 DNS 服务器也没有缓存，它会向上级 DNS 服务器查询，直到找到根 DNS 服务器。
- d. 最终，DNS 解析返回目标服务器的 IP 地址。

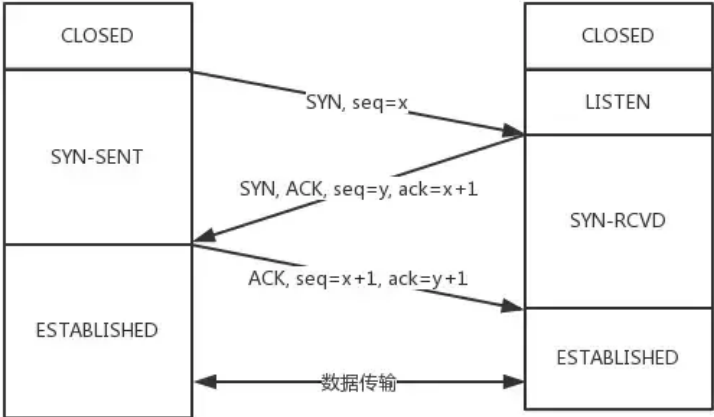
2. CDN

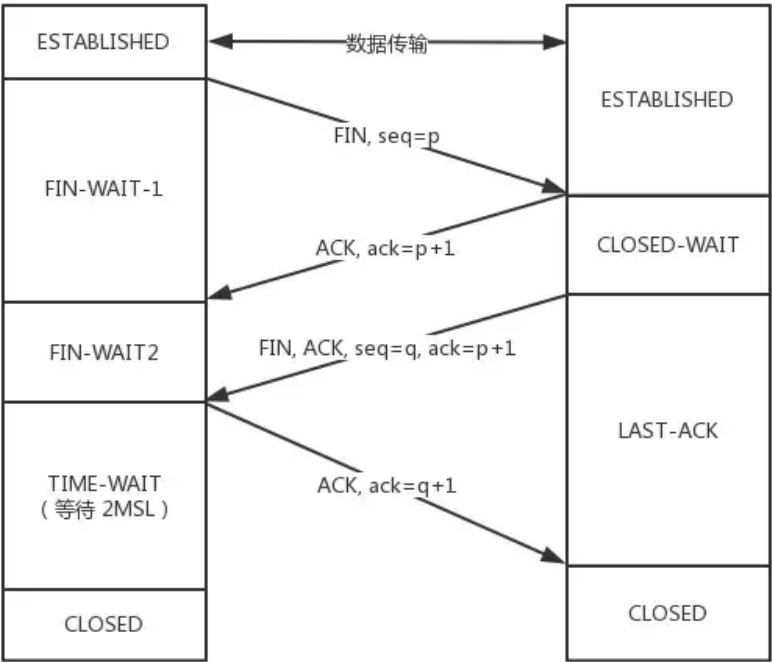
- a. DNS 服务器根据用户 IP 地址，将域名解析成相应节点的缓存服务器IP地址，实现用户就近访问

3. 建立网络连接

浏览器使用解析到的 IP 地址，与目标服务器建立网络连接：

- 1. **TCP 连接**：浏览器发起一个 TCP 三次握手过程，与服务器建立可靠的 TCP 连接。
- 2. **TLS/SSL 加密**：如果是 HTTPS 协议，浏览器和服务器会进行 TLS/SSL 握手，协商加密密钥，确保数据传输的安全。





4. 发送 HTTP/HTTPS 请求

浏览器通过已建立的连接，向服务器发送 HTTP 或 HTTPS 请求。请求包括：

- **请求行**：例如 `GET / HTTP/1.1`，表示请求根路径。
- **请求头**：包含浏览器信息（User-Agent）、语言偏好、请求来源（Referer）等。
- **请求体**（如果适用）：例如在 POST 请求中，包含表单数据或其他内容。

5. 服务器处理请求

服务器接收到请求后，根据请求内容进行处理：

6. 浏览器接收响应

服务器将响应通过网络发送回浏览器：

1. 浏览器接收完整的 HTTP 响应。
2. 浏览器解析响应头，根据 `Content-Type` 确定响应内容的类型（如 HTML、JSON、图片等）。

7. 渲染页面

如果响应内容是 HTML 页面，浏览器会进入渲染阶段：

1. **解析 HTML**：浏览器的渲染引擎（如 Blink、Gecko）解析 HTML 文档，构建 **DOM（文档对象模型）树**。
2. **解析 CSS**：浏览器解析页面中的 CSS 样式表，构建 **CSSOM（CSS 对象模型）树**。
3. **构建渲染树**：将 DOM 树和 CSSOM 树结合，生成渲染树，确定页面的布局和样式。
4. **布局（Reflow）**：计算每个元素的位置和大小。
5. **绘制（Paint）**：将渲染树转换为屏幕上的像素，显示页面内容。

8. 加载资源

HTML 页面可能包含其他资源（如图片、JavaScript 文件、CSS 文件等）：

1. 浏览器会解析 HTML 中的资源链接（如 `<img>`、`<script>`、`<link>`）。
2. 浏览器会为每个资源发起新的 HTTP 请求，下载资源。
3. 如果是 JavaScript 文件，浏览器会解析并执行脚本。
4. 如果是 CSS 文件，浏览器会重新解析样式并更新渲染树。

9. 交互与动态加载

用户与页面交互（如点击按钮、输入内容）可能触发 JavaScript 代码执行：

1. JavaScript 可能通过 AJAX 或 Fetch API 发起异步请求，获取新数据。
2. 浏览器会根据 JavaScript 的操作动态更新 DOM，重新渲染页面的局部内容。

6. 完整的前端技术栈应该掌握到什么程度？



EncodeStudio
印客学院

印客学院2025匠心之作
内容&服务全面升级

WEB

Web前端

大厂工程师训练营

进阶前端架构
直击阿里P7



7. x厂面试参考标准解析

1. 面试注意点

- 聊项目（亮点/核心的项目）：
 - 负责的内容是什么；出发点（为啥要做）；什么都是自己觉得有亮点的事情，怎么做的（怎么思考/落地/规划的）；有没有什么实战的问题（着重难点），自己是怎么解决的；项目实际收益；具体某个模块是怎么做的
- 算法：
 - 有时候还可能不会考算法，具体要看面试官的风格，如果考的话类型也没有一定的，可以给的部分参考是coding相关部分/promise顺序/二叉树最近的父节点等
 - 纯技术问题（个例参考）：多线程如何顺序打印数字
- 风格：
 - i. 面试官会比较注重细节，关注一些技术细节的实现以及业务的情况。
 - ii. 谈项目，一方面看你是怎么实现的；另一方面了解你做的业务，看候选人的表达和理解能力。
 - iii. 主要聊项目，主要是项目设计 + 项目思考 + 技术选型
 - iv. 聊指标，重构结束的指标，和之前的相比，提升的原因具体是什么 (性能指标 + 业务数据指标)

2. 项目：印象深刻的问题是什么；核心关注什么

- a. 主要聊项目细节，场景。比较关注项目完成度，技术细节实现。（根据简历问）
- b. 换位思考：
 - i. 比如作为一个面试官，候选人要给好的级别的话，每轮面试肯定就要介绍项目最有亮点的事情，这一块只要好好准备，就能把项目讲的非常牛逼（自己说的话就经得起推敲）；
 - ii. 最起码准备两个项目，不管是从背景到做什么事情，都要了解细节到非常清楚，并且能支撑他这种级别的。有些候选人会留一些口子让面试官去提问，类似于工程化设计等，说的不是很清楚先一笔带过，其实是准备过的。
 - iii. 介绍项目的同时，带一点业务指标，特别是后面的效果，很多面试的后端人选不知道业务指标的。
 - iv. 例如做之前是怎么样，做之后是怎么样，可能还会问他一些，为什么要做ab测试，ab测试的指标什么的；ab测试是不是成本比较高。

3. 关注人选对业界对比的认知，核心对比：性能优化如何做的/性能关注哪些/核心项目讲解思路和做法；以及现在看机会的原因。

8. 手撕面试真实算法题

8.1 JSON转树 树转JSON

Json 转树

Code block

```
1 function convertToTree(data) {
2   // 创建节点映射表 (id -> 节点)
3   const nodeMap = new Map();
4
5   // 第一次遍历: 创建所有节点的映射并初始化
   children数组
6   data.forEach(item => {
7     nodeMap.set(item.id, { ...item,
8       children: [] });
9   });
10   const roots = []; // 存储根节点
11 }
```

树转json

Code block

```
1 function convertToJson(treeData) {
2   const result = [];
3
4   function traverse(node) {
5     // 创建当前节点的副本, 不包含children属性
6     const { children, ...rest } = node;
7     result.push(rest);
8
9     // 递归处理子节点
10    if (children && children.length > 0) {
11      children.forEach(child =>
12        traverse(child));
13    }
14  }
```

```
12 // 第二次遍历: 构建树结构
13 data.forEach(item => {
14     const node = nodeMap.get(item.id);
15     if (item.parentId === null) {
16         roots.push(node); // 根节点直接加入
17     } else {
18         // 找到父节点并将当前节点加入其
19         // children
20         const parent =
21         nodeMap.get(item.parentId);
22         if (parent) {
23             parent.children.push(node);
24         }
25     }
26 });
27
28 return roots;
29 }
30
31 时间复杂度 O(n):
32 两次独立的数组遍历 (O(n) + O(n) = O(n))
33 Map 的插入和查找操作均为 O(1)
34 空间复杂度 O(n):
35 使用 Map 存储所有节点 (空间 O(n))
36 结果树复用节点对象
```

```
13 }
14
15 // 遍历每棵树的根节点
16 treeData.forEach(root => traverse(root));
17
18 return result;
19 }
```

DFS 深度优先

1. 时间复杂度: $O(n)$, 其中 n 是树中所有节点的总数, 每个节点只被访问一次
2. 空间复杂度: $O(h)$, 其中 h 是树的最大深度, 这是递归调用的栈空间

8.2 前序中序后序排列

1. 时间复杂度: 所有实现均为 $O(n)$, 每个节点只被访问一次
2. 空间复杂度:
 - 递归实现: $O(h)$, h 为树的高度 (递归调用栈)
 - 迭代实现: $O(n)$, 最坏情况下需要存储所有节点

前序

```
Code block
1 // 递归
2 function
3 preOrderTraversal(root) {
4     const result = [];
5
6     function traverse(node)
7     {
8         if (!node) return;
9         result.push(node.value);
10        // 访问根节点
11        traverse(node.left);
12        // 遍历左子树
13        traverse(node.right);
14        // 遍历右子树
15    }
16    traverse(root);
17    return result;
18 }
```

中序

```
Code block
1 // 递归
2 function
3 inOrderTraversal(root) {
4     const result = [];
5
6     function traverse(node)
7     {
8         if (!node) return;
9         traverse(node.left);
10        // 遍历左子树
11        result.push(node.value);
12        // 访问根节点
13        traverse(node.right);
14        // 遍历右子树
15    }
16    traverse(root);
17    return result;
18 }
```

后序

```
Code block
1 // 递归
2 function
3 postOrderTraversal(root) {
4     const result = [];
5
6     function traverse(node)
7     {
8         if (!node) return;
9         traverse(node.left);
10        // 遍历左子树
11        traverse(node.right);
12        // 遍历右子树
13        result.push(node.value);
14        // 访问根节点
15    }
16    traverse(root);
17    return result;
18 }
```



```

17 function
preOrderTraversal(root) {
18   if (!root) return [];
19
20   const result = [];
21   const stack = [root];
22
23   while (stack.length) {
24     const node =
stack.pop();
25     result.push(node.value);
26
27     // 先压入右节点，再压入左
节点
28     if (node.right)
stack.push(node.right);
29     if (node.left)
stack.push(node.left);
30   }
31
32   return result;
33 }

```

```

17 function
inOrderTraversal(root) {
18   const result = [];
19   const stack = [];
20   let current = root;
21
22   while (current ||
stack.length) {
23     // 先遍历到最左节点
24     while (current) {
25       stack.push(current);
26       current =
current.left;
27     }
28
29     current = stack.pop();
30
31     result.push(current.value)
;
32     current =
current.right;
33   }
34   return result;
35 }

```

```

17 function
postOrderTraversal(root) {
18   if (!root) return [];
19
20   const result = [];
21   const stack = [root];
22   const visited = new
Set();
23
24   while (stack.length) {
25     const node =
stack[stack.length - 1];
26
27     // 如果左右子节点都已访问
过，则可以访问当前节点
28     if (
29       (node.left === null
&& node.right === null) ||
30       (visited.has(node.left)
&& visited.has(node.right)
31       ) {
32       result.push(node.value);
33       visited.add(node);
34       stack.pop();
35     } else {
36       // 先压入右节点，再压
入左节点
37       if (node.right &&
!visited.has(node.right))
stack.push(node.right);
38       if (node.left &&
!visited.has(node.left))
stack.push(node.left);
39     }
40   }
41
42   return result;
43 }

```

8.3 红绿灯

Code block

```

1 function red() {
2   console.log('红灯亮');
3 }
4 function yellow() {
5   console.log('黄灯亮');
6 }
7 function green() {
8   console.log('绿灯亮');
9 }
10

```

Code block

```

1 function light(callback, timer) {
2   return new Promise(resolve => {

```

Code block

```

1 async function lightStep() {
2   await light(red, 3000);

```



```

3         setTimeout(() => {
4             callback?.();
5             resolve();
6         }, timer);
7     })
8 }
9
10 function lightStep() {
11     Promise.resolve().then(() => {
12         return light(red, 3000);
13     }).then(() => {
14         return light(yellow, 2000);
15     }).then(() => {
16         return light(green, 1000);
17     }).finally(() => {
18         return lightStep();
19     });
20 }

```

```

3     await light(yellow, 2000);
4     await light(green, 1000);
5     await lightStep();
6 }
7

```

8.4 实现有并行限制的 Promise 调度器

Code block

```

1 class Scheduler {
2     constructor(max) {
3         // 最大可并发任务数
4         this.max = max;
5         // 当前并发任务数
6         this.count = 0;
7         // 阻塞的任务队列
8         this.queue = [];
9     }
10
11     async add(fn) {
12         if (this.count >= this.max) {
13             // 若当前正在执行的任务，达到最大容量max
14             // 阻塞在此处，等待前面的任务执行完毕后将
15             // resolve弹出并执行
16             await new Promise(resolve =>
17                 this.queue.push(resolve));
18             // 当前并发任务数++
19             this.count++;
20             // 使用await执行此函数
21             const res = await fn();
22             // 执行完毕，当前并发任务数--
23             this.count--;
24             // 若队列中有值，将其resolve弹出，并执行
25             // 以便阻塞的任务，可以正常执行
26             this.queue.length && this.queue.shift();
27             // 返回函数执行的结果
28             return res;
29         }
30     }
31 }

```

Code block

```

1 // 延迟函数
2 const sleep = time => new Promise(resolve =>
3     setTimeout(resolve, time));
4
5 // 同时进行的任务最多2个
6 const scheduler = new Scheduler(2);
7
8 // 添加异步任务
9 // time: 任务执行的时间
10 // val: 参数
11 const addTask = (time, val) => {
12     scheduler.add(() => {
13         return sleep(time).then(() =>
14             console.log(val));
15     });
16 }
17
18 addTask(1000, '1');
19 addTask(500, '2');
20 addTask(300, '3');
21 addTask(400, '4');
22
23 // 2
24 // 3
25 // 1
26 // 4

```

8.5 模拟大数相加

Code block

```
1  var addStrings = function (num1, num2) {
2    let l1 = num1.length - 1
3    let l2 = num2.length - 1
4    let add = 0
5    let total = ''
6    while (l1 >= 0 || l2 >= 0 || add) {
7      let t = (num1[l1] * 1 || 0) + (num2[l2] * 1 || 0) + add
8      total = t % 10 + total
9      add = t > 9 ? 1 : 0
10     l1--
11     l2--
12   }
13   return total
14 };
```

8.6 其他

1. 版本号判断
2. 数组合并 排序